

Section A: Conceptual Understanding

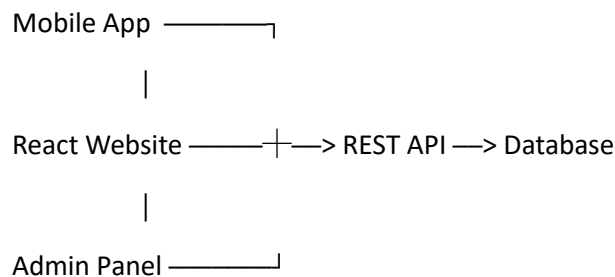
1. Why REST APIs are preferred for mobile and frontend-heavy apps

REST APIs are preferred because they allow the **frontend and backend to work independently**.

For example:

- Frontend can be built using React, Angular, Flutter, Android, or iOS.
- Backend can be built using Django REST Framework.
- They communicate using HTTP requests such as GET, POST, PUT, and DELETE.
- Data is usually transferred as **JSON**, which is easy for frontend applications to understand.
- The same API can be used by a website, Android app, and iOS app.

Example:



So, REST provides **flexibility, scalability, and separation between frontend and backend**.

2. How serializers act as a validation layer and custom field-level validation

In Django REST Framework, a serializer converts data between:

Python/Django Object ↔ JSON

But serializers also **validate incoming data before it reaches the database**.

For example, suppose a doctor has a field called experience.

We don't want the user to enter a negative value.

```
from rest_framework import serializers
```

```
class DoctorSerializer(serializers.ModelSerializer):
```

```
    class Meta:
```

```
        model = Doctor
```

```
        fields = '__all__'
```

```
    def validate_experience(self, value):
```

```
        if value < 0:
```

```
            raise serializers.ValidationError(
                "Experience cannot be negative."
            )
```

```
        return value
```

Here:

```
validate_experience()
```

is called **field-level validation**.

If the user sends:

```
{
    "name": "Dr. Ali",
    "experience": -5
}
```

the serializer rejects it instead of allowing invalid data into the database.

So serializers provide a **validation layer between the request and database**.

3. Importance of appropriate HTTP status codes — 201 Created vs 200 OK

HTTP status codes tell the client what happened with its request.

200 OK

Used when the request was successful but **did not create a new resource**.

Example:

GET /doctors/

Response:

200 OK

Or after successfully updating data:

PUT /doctors/5/

can return:

200 OK

201 Created

Used when a **new resource has been successfully created**.

Example:

POST /doctors/

If a new doctor is created:

201 Created

This distinction is important because it makes the API's response **clear and predictable** for frontend/mobile developers.

4. Why pagination is required when listing doctors

Imagine there are **100,000 doctors** in the database.

If we request:

GET /doctors/

and return all 100,000 doctors at once:

- Response becomes very large.
- Database has to retrieve a huge number of records.
- Server uses more memory.
- Network transfer becomes slower.
- Mobile applications may become slow.

Instead, we use pagination:

Page 1 → Doctors 1–10

Page 2 → Doctors 11–20

Page 3 → Doctors 21–30

For example:

GET /doctors/?page=2

Pagination reduces the amount of data retrieved and sent in each request.

It can therefore improve **response time, memory usage, network usage, and overall scalability**.

However, pagination alone doesn't magically make every database query cheap; proper database indexing and efficient queries are still important.

5. Benefits of ViewSets over APIView for rapid CRUD development

With APIView, we normally have to create separate methods/endpoints ourselves.

For example:

```
class DoctorView(APIView):
```

```
    def get(self, request):
```

```
        ...
```

```
    def post(self, request):
```

```
        ...
```

```
    def put(self, request, id):
```

```
        ...
```

```
    def delete(self, request, id):
```

```
        ...
```

With a ModelViewSet, Django REST Framework provides common CRUD operations for us.

```
from rest_framework.viewsets import ModelViewSet
```

```
class DoctorViewSet(ModelViewSet):
```

```
    queryset = Doctor.objects.all()
```

```
    serializer_class = DoctorSerializer
```

A ModelViewSet provides:

GET → list doctors

GET → retrieve one doctor

POST → create doctor

PUT → update doctor

PATCH → partially update doctor

DELETE → delete doctor

When combined with a router, URLs can also be generated automatically.

Therefore, ViewSets are useful when building **standard CRUD APIs quickly with less code.**

6. Why use `transaction.atomic()` when creating related doctor records?

Suppose creating a doctor requires creating **multiple related records**.

For example:

Doctor

↓

Doctor Profile

↓

Doctor Address

Imagine the program successfully creates the Doctor and Profile but fails while creating the Address.

Now the database could contain:

Doctor → Created 

Profile → Created 

Address → Failed 

This leaves the database in an incomplete state.

`transaction.atomic()` solves this by treating multiple database operations as **one transaction**.

```
from django.db import transaction
```

```
@transaction.atomic
```

```
def create_doctor(request):
```

```
    doctor = Doctor.objects.create(
        name="Dr. Ali"
    )
```

```
    profile = DoctorProfile.objects.create(
        doctor=doctor
    )
```

```
    address = DoctorAddress.objects.create(
        doctor=doctor
    )
```

return doctor

If everything succeeds:

Doctor 

Profile 

Address 

If something fails:

Doctor 

Profile 

Address 

The successful operations are rolled back.

So, `transaction.atomic()` helps maintain **data integrity and consistency**, especially when several related database records must be created together.